

CE 216: Fundamental Topics in Programming I

Spring 2026

Milestone 1: Design and Architecture Document Sports Manager Project

Instructor: Assoc. Prof. Dr. Kaya Oğuz

Group: Final Four

Name Surname	Student ID
Bilgenur Memiş	20240602050
Elif Güneş Tüzer	20240602072
Selim Baltacı	20230602011
Rabia Tülay Gökdağ	20210602029

March 15, 2026

Contents

1	Introduction	3
2	System Architecture	3
2.1	Core Goals	3
2.2	Layered Architecture	3
2.2.1	UI Layer	3
2.2.2	Game Logic Layer	4
2.2.3	Sport Layer	4
2.2.4	Data Layer	4
3	Core Classes	5
3.1	Interfaces and Sport Rules	5
3.1.1	ISport (Interface)	5
3.1.2	FootballSport	5
3.1.3	HeadballSport	5
3.2	Entity Hierarchy	5
3.2.1	BasePlayer (Abstract Class)	5
3.2.2	FootballPlayer and HeadballPlayer	6
3.2.3	BaseTeam	6
3.3	Simulation and Management	6
3.3.1	Match	6
3.3.2	MatchEngine	6
3.3.3	League	6
4	UML Class Diagram	7
4.1	UML Coordination and Technical Directives	7
5	User Interface Design	8
5.1	Key Screens	8
6	Extensibility	8
7	Conclusion	8

1 Introduction

Sports management games allow players to take the role of a team manager instead of directly controlling athletes during matches. In these games, the player makes decisions such as choosing the team lineup, managing players, and setting strategies that may affect the outcome of the matches. Because of this, the main challenge is not playing the sport itself but managing the team successfully throughout the season.

The goal of this project is to develop a Sports Manager application with a graphical user interface (GUI). The application will simulate a league environment where teams, players, and matches are organized within the system. Additionally, the application will use AI-generated data such as player and coach names, team names, and logos to make the game environment more realistic and visually engaging.

To support maximum flexibility, the project separates the user interface from the sport-specific logic. The interface interacts with the game through abstract structures and Java interfaces instead of depending on a specific sport implementation. This design allows new sports to be added without changing the user interface.

2 System Architecture

2.1 Core Goals

The framework is built to support multiple sports (like Volleyball or Headball) without the need to rewrite the entire system every time a new one is added.

- **Open for Extension:** New sports can be added easily without changing the core engine's code.
- **Separation of Concerns:** Sport rules, visuals (UI), and game logic are kept separate to prevent bugs and messy code.
- **High Extensibility:** Using standard Java tools (interfaces and abstract classes), new sports can be "plugged in" seamlessly to the existing structure.

2.2 Layered Architecture

The system follows a rigid four-layer structure to ensure modularity and facilitate independent development of game components.

2.2.1 UI Layer

The UI Layer is entirely decoupled from the business logic, interacting exclusively with the `ISport` interface to maintain sport-agnosticism. This layer supports specific data-binding

requirements, including the rendering of team logos and the management of AI-generated assets, such as names for players and coaches loaded from external sources.

2.2.2 Game Logic Layer

This layer governs the simulation lifecycle and competition organization through three primary components: the `MatchEngine` for outcome calculation, the `League` for automated fixture generation, and the `Match` class for tracking individual event execution and scores.

2.2.3 Sport Layer

The Sport Layer serves as the rule-set abstraction, centered on the `ISport` interface. This interface acts as a mandatory contract, forcing concrete implementations like `FootballSport` and `HeadballSport` to define critical parameters via specific methods.

2.2.4 Data Layer

This layer defines the persistence hierarchy and entity state. The `BasePlayer` abstract class serves as the entity foundation, encapsulating shared attributes including age, fitness, and injury status. Conversely, the `BaseTeam` class acts as a container for these entities.

3 Core Classes

This section describes the core classes of the Sports Manager application. The design relies heavily on abstraction and object-oriented principles to ensure flexibility and modularity.

3.1 Interfaces and Sport Rules

3.1.1 ISport (Interface)

The **ISport** interface defines the general structure and rules that every sport must follow. It follows the **Open-Closed Principle**, allowing new sports to be introduced without modifying the engine.

- **getRequiredPlayers()**: Returns the number of players required for a team.
- **getMatchDuration()**: Returns the duration of a match.
- **getPointForWin()**: Returns the points awarded for a win.

3.1.2 FootballSport

A concrete implementation of **ISport** defining football-specific rules, such as 11-player teams and standard durations. It remains independent of the UI and game management components.

3.1.3 HeadballSport

Another implementation of **ISport** representing a fictional sport. It demonstrates the system's flexibility as the game engine treats it identically to football via the interface contract.

3.2 Entity Hierarchy

3.2.1 BasePlayer (Abstract Class)

An abstract foundation for generic players. It cannot be instantiated directly and contains shared attributes:

- **name, age, fitness, isInjured**
- **Methods:** **train()** to improve fitness and **recover()** to handle fatigue/injury.

3.2.2 FootballPlayer and HeadballPlayer

These classes extend `BasePlayer`. While inheriting common attributes, they allow for sport-specific properties like tactical roles or skill ratings.

3.2.3 BaseTeam

Acts as a container for multiple player entities. It manages the `teamName`, the roster list, and total league points.

3.3 Simulation and Management

3.3.1 Match

Represents a game between two teams. It tracks `homeScore`, `awayScore`, and the `isPlayed` status. The `playMatch()` method triggers the simulation process.

3.3.2 MatchEngine

The core simulation component. It determines outcomes based on team strength, fitness, and sport rules using the `simulateMatch(team1, team2, rules)` method.

3.3.3 League

The central manager of the competition. It maintains the list of teams, generates the schedule via `generateFixture()`, and updates rankings through `updateStandings()`.

4 UML Class Diagram

The following diagram shows the system architecture and class interactions.

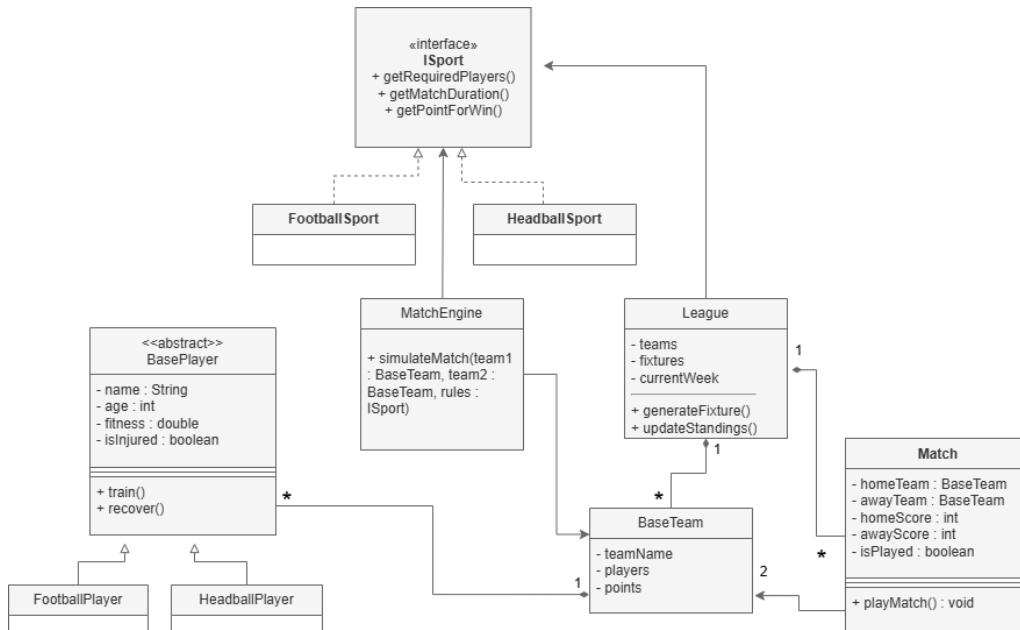


Figure 1: UML Class Diagram of the Sports Manager Framework

4.1 UML Coordination and Technical Directives

- **Interfaces:** The `ISport` interface is designated with the `«interface»` stereotype.
- **Abstract Classes:** The `BasePlayer` class is identified as abstract via italics.
- **Inheritance:** Solid arrows with empty triangular heads show inheritance from `BasePlayer`.
- **Composition/Aggregation:** The `League` aggregates `BaseTeam` instances and maintains composition with `Match` entities.

5 User Interface Design

The UI is designed for clarity and usability. It is completely separate from the core game logic and interacts only through the `ISport` interface and system classes.

5.1 Key Screens

- **Main Menu:** Launch screen to start games or select sports.
- **Team Screen:** Displays team info, logos, and squad data from `BaseTeam`.
- **League Table:** Shows updated standings managed by the `League` class.
- **Match Screen:** Displays scores calculated by the `MatchEngine`.
- **Player List:** Lists player attributes from classes extending `BasePlayer`.

6 Extensibility

Since the game logic only communicates through the `ISport` interface, adding a new sport like Volleyball is as simple as writing a new class that implements the interface. Nothing else in the system needs to change.

7 Conclusion

This project builds a flexible simulation by layering the UI, logic, and rules. The result is a clean, maintainable foundation that follows OOP principles and can grow as the project develops.